

The data are imported as csv and is analysed carefully using head, shape, info and describe.

```
In [2]: import pandas as pd

aqi = pd.read_csv("city_day.csv")
crop = pd.read_csv("crop_production.csv")

aqi.head()
```

Out[2]:

	City	Date	PM2.5	PM10	NO	NO2	NOx	NH3	CO	SO2	O3	B
0	Ahmedabad	2015-01-01	NaN	NaN	0.92	18.22	17.15	NaN	0.92	27.64	133.36	
1	Ahmedabad	2015-01-02	NaN	NaN	0.97	15.69	16.46	NaN	0.97	24.55	34.06	
2	Ahmedabad	2015-01-03	NaN	NaN	17.40	19.30	29.70	NaN	17.40	29.07	30.70	
3	Ahmedabad	2015-01-04	NaN	NaN	1.70	18.48	17.97	NaN	1.70	18.59	36.08	
4	Ahmedabad	2015-01-05	NaN	NaN	22.10	21.42	37.76	NaN	22.10	39.33	39.31	

```
In [3]: crop.head()
```

Out[3]:

	State_Name	District_Name	Crop_Year	Season	Crop	Area	Production
0	Andaman and Nicobar Islands	NICOBARS	2000	Kharif	Arecanut	1254.0	2000.0
1	Andaman and Nicobar Islands	NICOBARS	2000	Kharif	Other Kharif pulses	2.0	1.0
2	Andaman and Nicobar Islands	NICOBARS	2000	Kharif	Rice	102.0	321.0
3	Andaman and Nicobar Islands	NICOBARS	2000	Whole Year	Banana	176.0	641.0
4	Andaman and Nicobar Islands	NICOBARS	2000	Whole Year	Cashewnut	720.0	165.0

```
In [4]: aqi.shape
```

Out[4]: (29531, 16)

```
In [5]: crop.shape
```

Out[5]: (246091, 7)

In [6]: `print(aqi.columns)`

`print(crop.columns)`

```
Index(['City', 'Date', 'PM2.5', 'PM10', 'NO', 'NO2', 'NOx', 'NH3', 'CO', 'SO2',
      'O3', 'Benzene', 'Toluene', 'Xylene', 'AQI', 'AQI_Bucket'],
      dtype='object')
Index(['State_Name', 'District_Name', 'Crop_Year', 'Season', 'Crop', 'Area',
      'Production'],
      dtype='object')
```

In [7]: `print(aqi.info())`

`print(crop.info())`

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 29531 entries, 0 to 29530
Data columns (total 16 columns):
#   Column          Non-Null Count  Dtype
---  ---
0   City             29531 non-null  object
1   Date             29531 non-null  object
2   PM2.5            24933 non-null  float64
3   PM10             18391 non-null  float64
4   NO               25949 non-null  float64
5   NO2              25946 non-null  float64
6   NOx              25346 non-null  float64
7   NH3              19203 non-null  float64
8   CO               27472 non-null  float64
9   SO2              25677 non-null  float64
10  O3               25509 non-null  float64
11  Benzene          23908 non-null  float64
12  Toluene          21490 non-null  float64
13  Xylene           11422 non-null  float64
14  AQI              24850 non-null  float64
15  AQI_Bucket       24850 non-null  object
dtypes: float64(13), object(3)
memory usage: 3.6+ MB
None
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 246091 entries, 0 to 246090
Data columns (total 7 columns):
#   Column          Non-Null Count  Dtype
---  ---
0   State_Name       246091 non-null  object
1   District_Name    246091 non-null  object
2   Crop_Year        246091 non-null  int64
3   Season           246091 non-null  object
4   Crop             246091 non-null  object
5   Area             246091 non-null  float64
6   Production       242361 non-null  float64
dtypes: float64(2), int64(1), object(4)
memory usage: 13.1+ MB
None
```

In [8]: `print(aqi.describe())`

`print(crop.describe())`

	PM2.5	PM10	NO	NO2	NOx \
count	24933.000000	18391.000000	25949.000000	25946.000000	25346.000000
mean	67.450578	118.127103	17.574730	28.560659	32.309123
std	64.661449	90.605110	22.785846	24.474746	31.646011
min	0.040000	0.010000	0.020000	0.010000	0.000000
25%	28.820000	56.255000	5.630000	11.750000	12.820000
50%	48.570000	95.680000	9.890000	21.690000	23.520000
75%	80.590000	149.745000	19.950000	37.620000	40.127500
max	949.990000	1000.000000	390.680000	362.210000	467.630000

	NH3	CO	SO2	O3	Benzene \
count	19203.000000	27472.000000	25677.000000	25509.000000	23908.000000
mean	23.483476	2.248598	14.531977	34.491430	3.280840
std	25.684275	6.962884	18.133775	21.694928	15.811136
min	0.010000	0.000000	0.010000	0.010000	0.000000
25%	8.580000	0.510000	5.670000	18.860000	0.120000
50%	15.850000	0.890000	9.160000	30.840000	1.070000
75%	30.020000	1.450000	15.220000	45.570000	3.080000
max	352.890000	175.810000	193.860000	257.730000	455.030000

	Toluene	Xylene	AQI
count	21490.000000	11422.000000	24850.000000
mean	8.700972	3.070128	166.463581
std	19.969164	6.323247	140.696585
min	0.000000	0.000000	13.000000
25%	0.600000	0.140000	81.000000
50%	2.970000	0.980000	118.000000
75%	9.150000	3.350000	208.000000
max	454.850000	170.370000	2049.000000

	Crop_Year	Area	Production
count	246091.000000	2.460910e+05	2.423610e+05
mean	2005.643018	1.200282e+04	5.825034e+05
std	4.952164	5.052340e+04	1.706581e+07
min	1997.000000	4.000000e-02	0.000000e+00
25%	2002.000000	8.000000e+01	8.800000e+01
50%	2006.000000	5.820000e+02	7.290000e+02
75%	2010.000000	4.392000e+03	7.023000e+03
max	2015.000000	8.580100e+06	1.250800e+09

.isnull().sum() will get the sum of missing values.

```
In [9]: print(aqi.isnull().sum())

        print(crop.isnull().sum())
```

```

City          0
Date          0
PM2.5        4598
PM10         11140
NO           3582
NO2          3585
NOx          4185
NH3          10328
CO           2059
SO2          3854
O3           4022
Benzene       5623
Toluene       8041
Xylene       18109
AQI          4681
AQI_Bucket    4681
dtype: int64
State_Name    0
District_Name 0
Crop_Year     0
Season        0
Crop          0
Area          0
Production    3730
dtype: int64
State_Name    0
District_Name 0
Crop_Year     0
Season        0
Crop          0
Area          0
Production    3730
dtype: int64

```

the following code is to check whether any duplicated rows exist.

```

In [10]: print(aqi.duplicated().sum())

print(crop.duplicated().sum())

```

```

0
0

```

Task 2

The following code gives the percentage of missing values for each column of AQI dataset.

```

In [11]: missing_summary = pd.DataFrame({
    'Missing Count': aqi.isnull().sum(),
    'Missing %': round(aqi.isnull().mean()*100,2)
})

missing_summary.sort_values('Missing %', ascending=False)

```

Out[11]:

	Missing Count	Missing %
Xylene	18109	61.32
PM10	11140	37.72
NH3	10328	34.97
Toluene	8041	27.23
Benzene	5623	19.04
AQI	4681	15.85
AQI_Bucket	4681	15.85
PM2.5	4598	15.57
NOx	4185	14.17
O3	4022	13.62
SO2	3854	13.05
NO2	3585	12.14
NO	3582	12.13
CO	2059	6.97
Date	0	0.00
City	0	0.00

Skewness measures the asymmetry of a distribution. Skewness = 0 - Perfectly symmetric distribution. Positive Skew (> 0) and Negative Skew (< 0) says that : Outliers are present. Mean can be misleading.

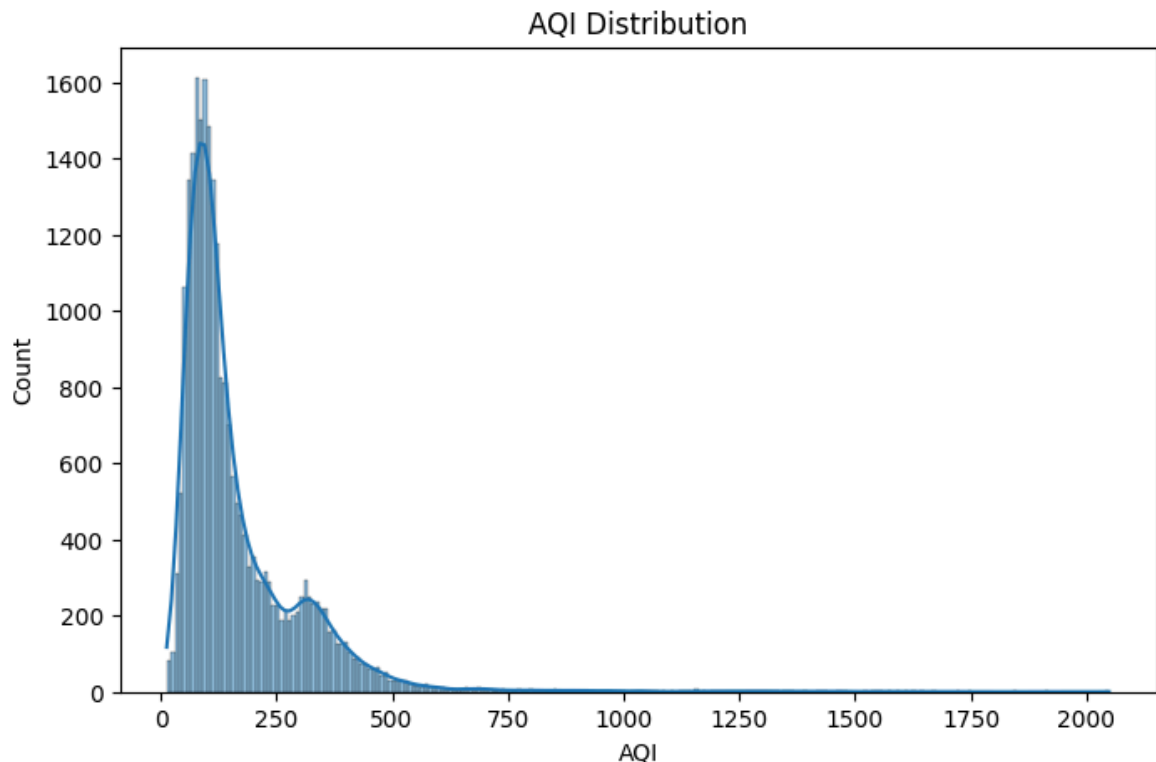
```
In [12]: aqi[['PM2.5', 'PM10', 'NO', 'NO2', 'NOx',
              'NH3', 'CO', 'SO2', 'O3',
              'Benzene', 'Toluene', 'Xylene', 'AQI']].skew()
```

```
Out[12]: PM2.5      3.369960
          PM10      2.053189
          NO       3.883166
          NO2      2.464560
          NOx      2.569915
          NH3      4.083993
          CO       8.878322
          SO2      4.083660
          O3       1.330119
          Benzene  21.304218
          Toluene  11.666129
          Xylene   7.891515
          AQI      3.396757
          dtype: float64
```

```
In [13]: import seaborn as sns
          import matplotlib.pyplot as plt

          plt.figure(figsize=(8,5))
```

```
sns.histplot(aqi['AQI'], kde=True)
plt.title("AQI Distribution")
plt.show()
```



Deciding to drop Xylene as 61% missing values will lead to no meaningful output. About AQI column, it is decided to drop only missing rows as it is only 15%

```
In [14]: aqi_clean = aqi.copy()

aqi_clean = aqi_clean.drop(columns=['Xylene'])

aqi_clean = aqi_clean.dropna(subset=['AQI'])
```

```
In [15]: aqi_clean['AQI_Bucket'].isnull().sum()
```

```
Out[15]: np.int64(0)
```

Rest all the columns are decided to be filled with median as the skew is not 0, so mean is not possible and median is the next option.

```
In [16]: num_cols = [
    'PM2.5', 'PM10', 'NO', 'NO2', 'NOx',
    'NH3', 'CO', 'SO2', 'O3',
    'Benzene', 'Toluene'
]

for col in num_cols:
    aqi_clean[col] = aqi_clean[col].fillna(
        aqi_clean[col].median()
    )
```

```
In [17]: aqi_clean.isnull().sum()
```

```
Out[17]: City          0
         Date          0
         PM2.5         0
         PM10          0
         NO            0
         NO2           0
         NOx           0
         NH3           0
         CO            0
         SO2           0
         O3            0
         Benzene       0
         Toluene       0
         AQI           0
         AQI_Bucket    0
         dtype: int64
```

```
In [18]: aqi_clean.shape
```

```
Out[18]: (24850, 15)
```

Task 3

```
In [19]: crop.isnull().sum()
```

```
Out[19]: State_Name      0
         District_Name   0
         Crop_Year       0
         Season          0
         Crop            0
         Area            0
         Production      3730
         dtype: int64
```

```
In [20]: crop['Production'] = crop['Production'].fillna(
         crop['Production'].median()
         )
```

The following output shows the list of cities in AQI dataset. this is to map the cities to its corresponding states.

```
In [21]: sorted(aqi_clean['City'].unique())
```

```
Out[21]: ['Ahmedabad',  
          'Aizawl',  
          'Amaravati',  
          'Amritsar',  
          'Bengaluru',  
          'Bhopal',  
          'Brajrajnagar',  
          'Chandigarh',  
          'Chennai',  
          'Coimbatore',  
          'Delhi',  
          'Ernakulam',  
          'Gurugram',  
          'Guwahati',  
          'Hyderabad',  
          'Jaipur',  
          'Jorapokhar',  
          'Kochi',  
          'Kolkata',  
          'Lucknow',  
          'Mumbai',  
          'Patna',  
          'Shillong',  
          'Talcher',  
          'Thiruvananthapuram',  
          'Visakhapatnam']
```

```
In [22]: city_to_state = {  
          'Ahmedabad': 'Gujarat',  
          'Aizawl': 'Mizoram',  
          'Amaravati': 'Andhra Pradesh',  
          'Amritsar': 'Punjab',  
          'Bengaluru': 'Karnataka',  
          'Bhopal': 'Madhya Pradesh',  
          'Brajrajnagar': 'Odisha',  
          'Chandigarh': 'Chandigarh',  
          'Chennai': 'Tamil Nadu',  
          'Coimbatore': 'Tamil Nadu',  
          'Delhi': 'Delhi',  
          'Ernakulam': 'Kerala',  
          'Gurugram': 'Haryana',  
          'Guwahati': 'Assam',  
          'Hyderabad': 'Telangana',  
          'Jaipur': 'Rajasthan',  
          'Jorapokhar': 'Jharkhand',  
          'Kochi': 'Kerala',  
          'Kolkata': 'West Bengal',  
          'Lucknow': 'Uttar Pradesh',  
          'Mumbai': 'Maharashtra',  
          'Patna': 'Bihar',  
          'Shillong': 'Meghalaya',  
          'Talcher': 'Odisha',  
          'Thiruvananthapuram': 'Kerala',  
          'Visakhapatnam': 'Andhra Pradesh'  
        }
```

```
In [23]: aqi_clean['State'] = aqi_clean['City'].map(city_to_state)
```

```
In [24]: aqi_clean['State'].isnull().sum()
```

Out[24]: np.int64(0)

The following code is to remove spaces and format the names of states capitalised and uniform.

```
In [25]: aqi_clean['State'] = aqi_clean['State'].str.strip()
crop['State_Name'] = crop['State_Name'].str.strip()
```

```
In [26]: aqi_clean['State'] = aqi_clean['State'].str.title()
crop['State_Name'] = crop['State_Name'].str.title()
```

```
In [27]: sorted(crop['State_Name'].unique())
```

```
Out[27]: ['Andaman And Nicobar Islands',
'Andhra Pradesh',
'Arunachal Pradesh',
'Assam',
'Bihar',
'Chandigarh',
'Chhattisgarh',
'Dadra And Nagar Haveli',
'Goa',
'Gujarat',
'Haryana',
'Himachal Pradesh',
'Jammu And Kashmir',
'Jharkhand',
'Karnataka',
'Kerala',
'Madhya Pradesh',
'Maharashtra',
'Manipur',
'Meghalaya',
'Mizoram',
'Nagaland',
'Odisha',
'Puducherry',
'Punjab',
'Rajasthan',
'Sikkim',
'Tamil Nadu',
'Telangana',
'Tripura',
'Uttar Pradesh',
'Uttarakhand',
'West Bengal']
```

The following Output proves that there are some values that do not exist in both dataset simultaneously.

```
In [28]: aqi_states = set(aqi_clean['State'].unique())
crop_states = set(crop['State_Name'].unique())

print("Only in AQI:")
print(aqi_states - crop_states)
```

```
print("\nOnly in Crop:")
print(crop_states - aqi_states)
```

Only in AQI:
{ 'Delhi' }

Only in Crop:
{ 'Dadra And Nagar Haveli', 'Tripura', 'Nagaland', 'Chhattisgarh', 'Arunachal Pradesh', 'Jammu And Kashmir', 'Goa', 'Puducherry', 'Andaman And Nicobar Islands', 'Himachal Pradesh', 'Sikkim', 'Manipur', 'Uttarakhand' }

```
In [29]: print(crop['State_Name'].str.startswith(' ').sum())
print(crop['State_Name'].str.endswith(' ').sum())
```

0
0

Task 4

The majority of AQI observations are concentrated between approximately 50 and 250, indicating that most cities experience moderate to poor air quality rather than extremely severe pollution.

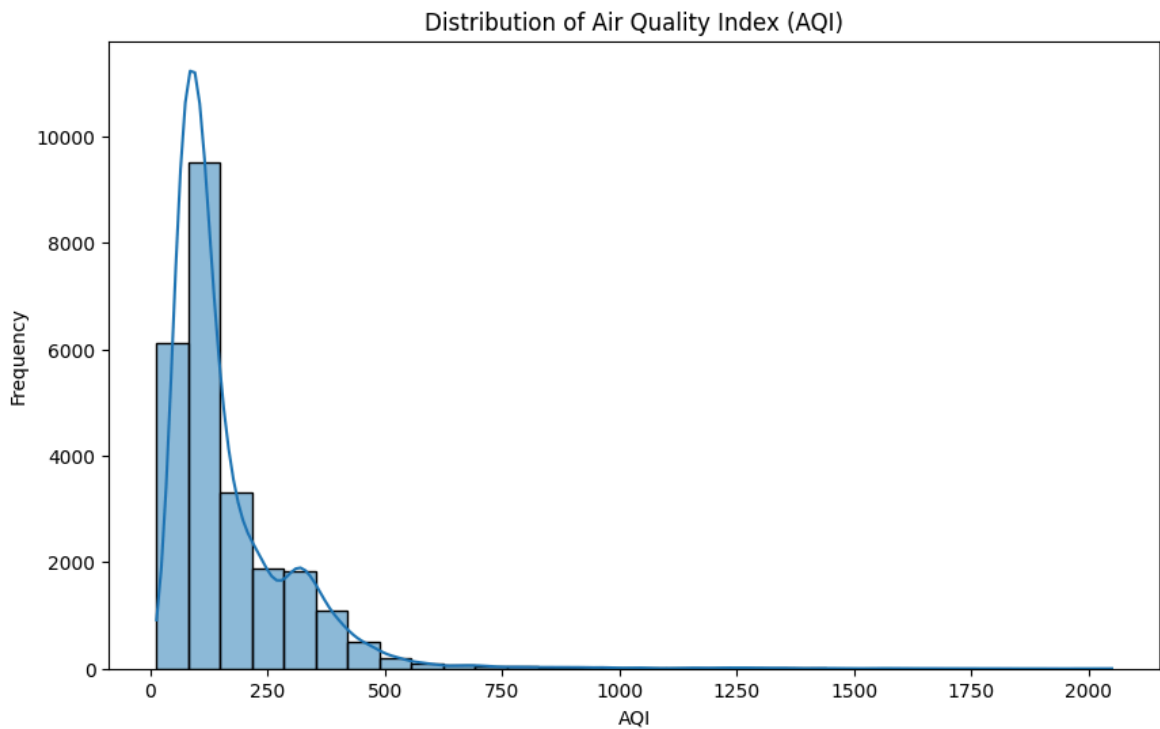
```
In [30]: import seaborn as sns
import matplotlib.pyplot as plt

plt.figure(figsize=(10,6))

sns.histplot(
    aqi_clean['AQI'],
    bins=30,
    kde=True
)

plt.title("Distribution of Air Quality Index (AQI)")
plt.xlabel("AQI")
plt.ylabel("Frequency")

plt.show()
```



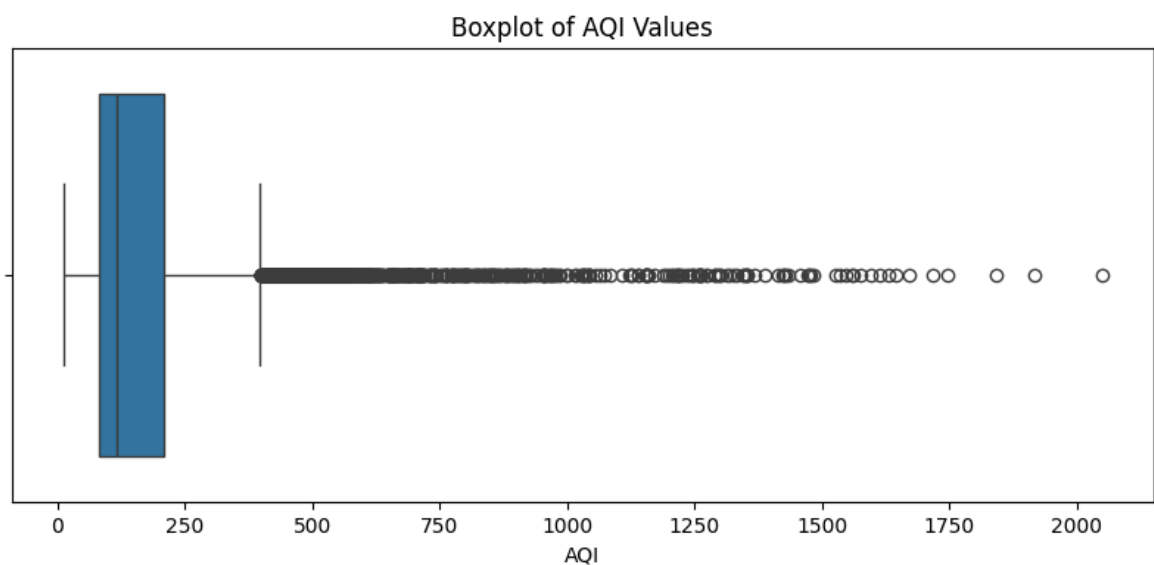
A large number of observations lie beyond the upper whisker, with AQI values extending above 2000. These extreme values contribute to the strong positive skewness of the distribution and may inflate the average AQI.

```
In [31]: plt.figure(figsize=(10,4))

sns.boxplot(
    x=aqi_clean['AQI']
)

plt.title("Boxplot of AQI Values")
plt.xlabel("AQI")

plt.show()
```



```
In [32]: print("Mean AQI :", round(aqi_clean['AQI'].mean(),2))
print("Median AQI :", round(aqi_clean['AQI'].median(),2))
```

Mean AQI : 166.46
Median AQI : 118.0

If mean is much higher than median, that directly proves Extreme values are pulling the average upward.

Task 5

25% of AQI values are below 81 75% of AQI values are below 208

```
In [33]: Q1 = aqi_clean['AQI'].quantile(0.25)

Q3 = aqi_clean['AQI'].quantile(0.75)

IQR = Q3 - Q1

lower_bound = Q1 - 1.5 * IQR

upper_bound = Q3 + 1.5 * IQR

print("Q1:", Q1)
print("Q3:", Q3)
print("IQR:", IQR)
print("Lower Bound:", lower_bound)
print("Upper Bound:", upper_bound)
```

Q1: 81.0
Q3: 208.0
IQR: 127.0
Lower Bound: -109.5
Upper Bound: 398.5

So about 5.5% of AQI readings are unusually high.

```
In [34]: outliers = aqi_clean[
    (aqi_clean['AQI'] < lower_bound) |
    (aqi_clean['AQI'] > upper_bound)
]

print("Number of Outliers:", len(outliers))
```

Number of Outliers: 1358

```
In [35]: outliers['AQI'].describe()
```

```
Out[35]: count    1358.000000
mean      565.407216
std       240.850834
min       399.000000
25%       428.000000
50%       472.000000
75%       576.000000
max       2049.000000
Name: AQI, dtype: float64
```

Possibility 1

Ahmedabad genuinely had severe pollution episodes. or Possibility 2

There may be data quality issues for Ahmedabad.

```
In [36]: outliers[['City', 'Date', 'AQI']].sort_values(
            by='AQI',
            ascending=False
        ).head(20)
```

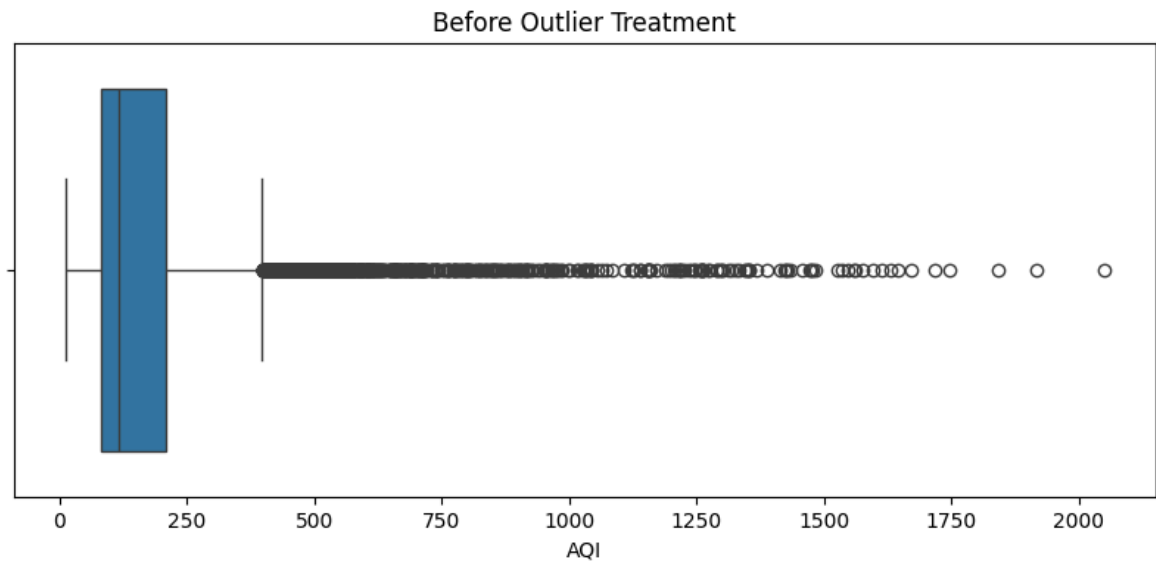
```
Out[36]:
```

	City	Date	AQI
1145	Ahmedabad	2018-02-19	2049.0
1136	Ahmedabad	2018-02-10	1917.0
560	Ahmedabad	2016-07-14	1842.0
1048	Ahmedabad	2017-11-14	1747.0
1463	Ahmedabad	2019-01-03	1719.0
1417	Ahmedabad	2018-11-18	1672.0
1420	Ahmedabad	2018-11-21	1646.0
1407	Ahmedabad	2018-11-08	1630.0
1127	Ahmedabad	2018-02-01	1613.0
1746	Ahmedabad	2019-10-13	1595.0
1047	Ahmedabad	2017-11-13	1577.0
1391	Ahmedabad	2018-10-23	1561.0
1451	Ahmedabad	2018-12-22	1558.0
1412	Ahmedabad	2018-11-13	1548.0
1481	Ahmedabad	2019-01-21	1537.0
1149	Ahmedabad	2018-02-23	1526.0
1148	Ahmedabad	2018-02-22	1483.0
1144	Ahmedabad	2018-02-18	1477.0
1461	Ahmedabad	2019-01-01	1474.0
1418	Ahmedabad	2018-11-19	1472.0

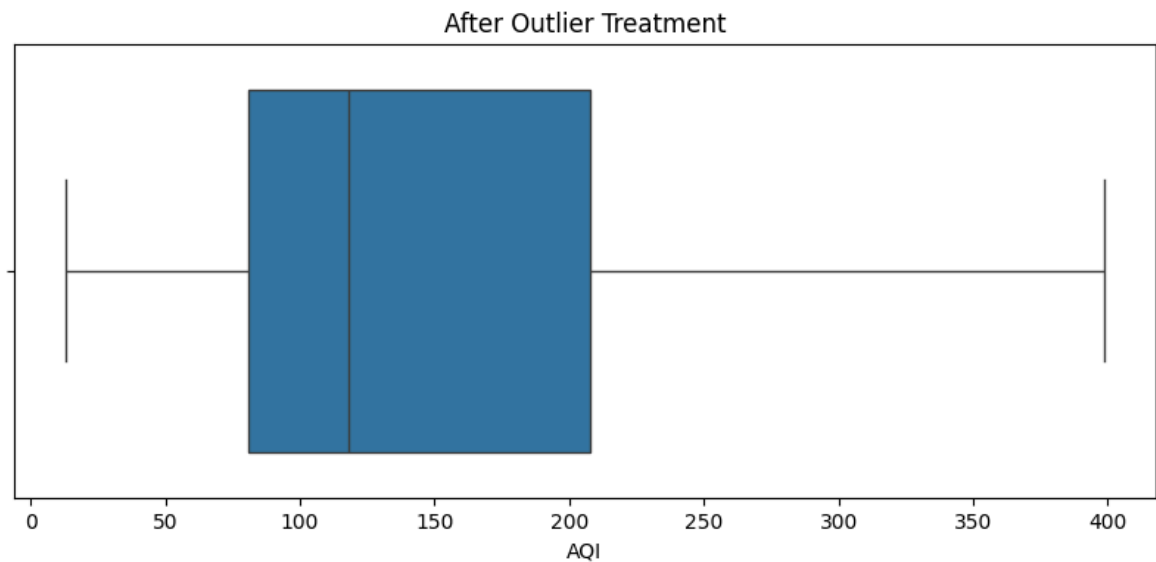
```
In [37]: aqi_treated = aqi_clean.copy()

aqi_treated['AQI'] = aqi_treated['AQI'].clip(
    upper=upper_bound
)
```

```
In [38]: plt.figure(figsize=(10,4))
sns.boxplot(x=aqi_clean['AQI'])
plt.title("Before Outlier Treatment")
plt.show()
```



```
In [39]: plt.figure(figsize=(10,4))
sns.boxplot(x=aqi_treated['AQI'])
plt.title("After Outlier Treatment")
plt.show()
```



Instead of removing observations, winsorization (capping) was applied using the IQR upper bound (398.5). This approach reduced the influence of extreme values while preserving all records for analysis.

Lab 2

Task 6

```
In [40]: aqi_clean['Date']
```

```
Out[40]: 28      2015-01-29
          29      2015-01-30
          30      2015-01-31
          31      2015-02-01
          32      2015-02-02
          ...
          29526    2020-06-27
          29527    2020-06-28
          29528    2020-06-29
          29529    2020-06-30
          29530    2020-07-01
          Name: Date, Length: 24850, dtype: object
```

```
In [41]: aqi_clean['Date'] = pd.to_datetime(aqi_clean['Date'])
```

```
In [42]: aqi_clean['Date'].head()
```

```
Out[42]: 28      2015-01-29
          29      2015-01-30
          30      2015-01-31
          31      2015-02-01
          32      2015-02-02
          Name: Date, dtype: datetime64[ns]
```

```
In [43]: aqi_clean['Year'] = aqi_clean['Date'].dt.year
```

```
In [44]: aqi_clean[['Date', 'Year']].head()
```

```
Out[44]:
```

	Date	Year
28	2015-01-29	2015
29	2015-01-30	2015
30	2015-01-31	2015
31	2015-02-01	2015
32	2015-02-02	2015

```
In [45]: yearly_aqi = aqi_clean.groupby('Year')['AQI'].mean().reset_index()

yearly_aqi
```

```
Out[45]:
```

	Year	AQI
0	2015	212.463054
1	2016	197.150019
2	2017	181.472789
3	2018	182.684312
4	2019	156.518173
5	2020	113.520697

```
In [46]: most_polluted = yearly_aqi.loc[
        yearly_aqi['AQI'].idxmax()
        ]

        print(most_polluted)
```

```
Year    2015.000000
AQI      212.463054
Name: 0, dtype: float64
```

```
In [47]: least_polluted = yearly_aqi.loc[
        yearly_aqi['AQI'].idxmin()
        ]

        print(least_polluted)
```

```
Year    2020.000000
AQI      113.520697
Name: 5, dtype: float64
```

```
In [48]: import matplotlib.pyplot as plt
import seaborn as sns

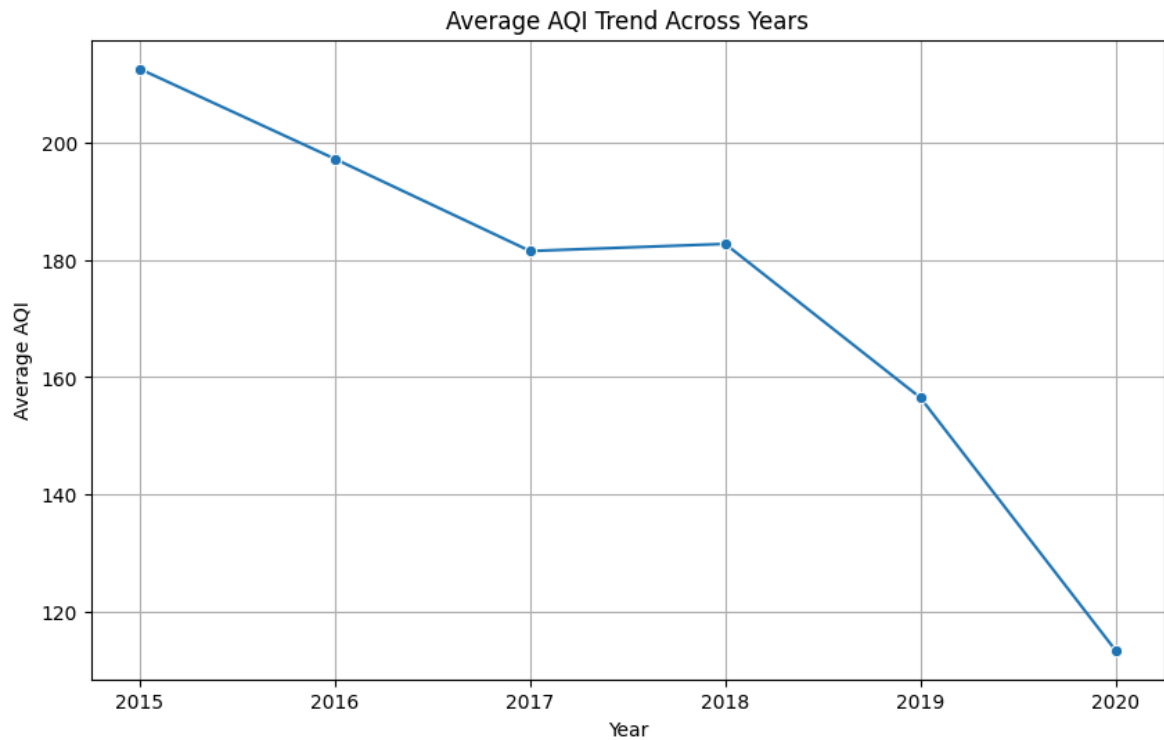
plt.figure(figsize=(10,6))

sns.lineplot(
    data=yearly_aqi,
    x='Year',
    y='AQI',
    marker='o'
)

plt.title('Average AQI Trend Across Years')
plt.xlabel('Year')
plt.ylabel('Average AQI')

plt.grid(True)

plt.show()
```



```
In [49]: plt.figure(figsize=(10,6))

sns.lineplot(
    data=yearly_aqi,
    x='Year',
    y='AQI',
    marker='o'
)

plt.scatter(
    most_polluted['Year'],
    most_polluted['AQI'],
    s=150
)

plt.scatter(
    least_polluted['Year'],
    least_polluted['AQI'],
    s=150
)

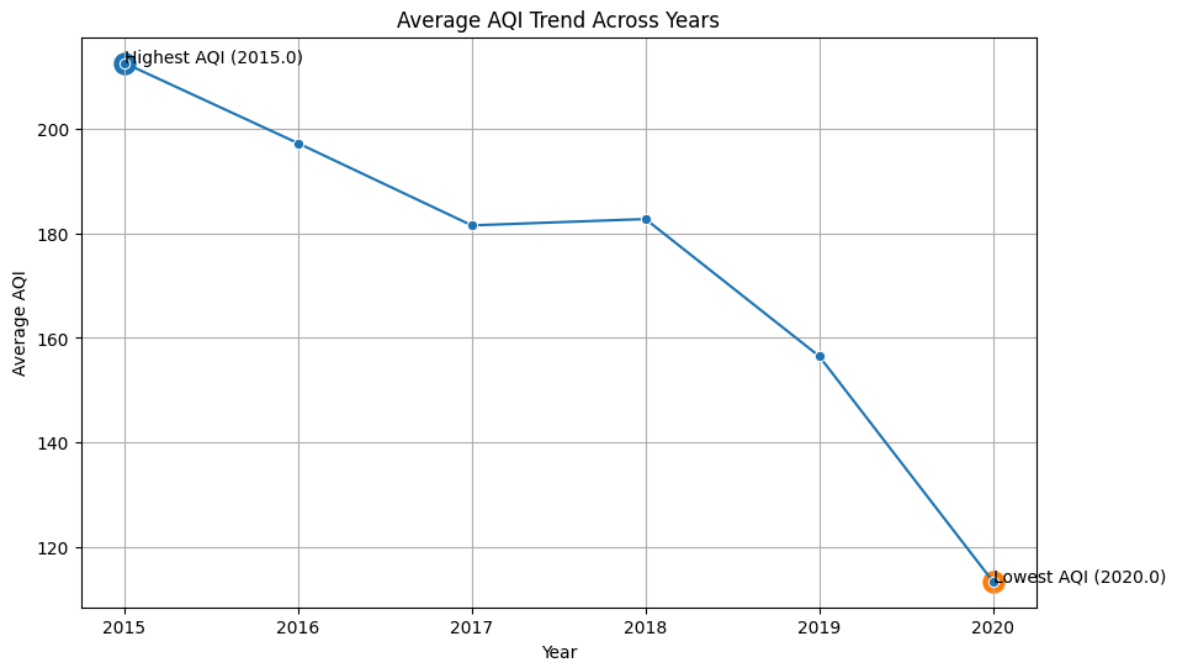
plt.annotate(
    f"Highest AQI ({most_polluted['Year']})",
    (most_polluted['Year'], most_polluted['AQI'])
)

plt.annotate(
    f"Lowest AQI ({least_polluted['Year']})",
    (least_polluted['Year'], least_polluted['AQI'])
)

plt.title('Average AQI Trend Across Years')
plt.xlabel('Year')
plt.ylabel('Average AQI')

plt.grid(True)
```

```
plt.show()
```



Air quality improved.

Line charts are the most effective visualisation for displaying trends across consecutive years

Task 7

Extract month from Date → Calculate average AQI for each month → Visualize it → Check whether Oct-Nov-Dec are actually the highest months → Conclude based on evidence

```
In [50]: aqi_clean['Month'] = aqi_clean['Date'].dt.month
```

```
In [51]: aqi_clean['Month_Name'] = aqi_clean['Date'].dt.month_name()
```

```
In [52]: monthly_aqi = aqi_clean.groupby('Month')['AQI'].mean().reset_index()

monthly_aqi
```

Out[52]:

	Month	AQI
0	1	231.674918
1	2	202.905197
2	3	164.735281
3	4	143.355120
4	5	135.489579
5	6	120.198379
6	7	111.854575
7	8	113.613176
8	9	115.191804
9	10	188.613552
10	11	241.681302
11	12	227.084980

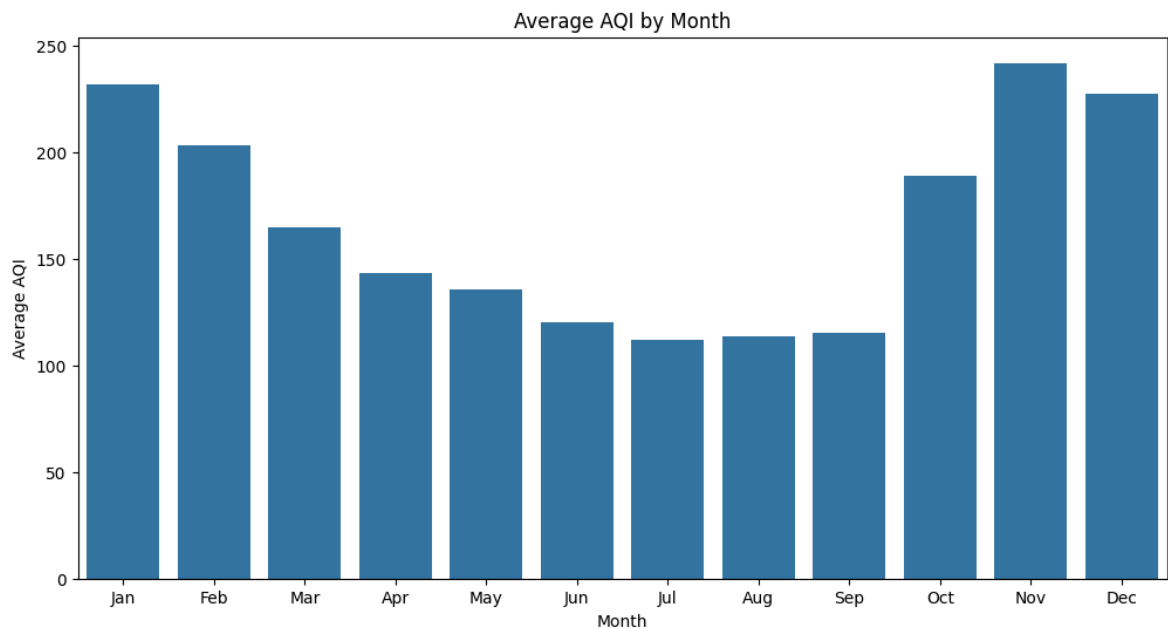
```
In [53]: monthly_aqi['Month_Name'] = [
        'Jan', 'Feb', 'Mar', 'Apr',
        'May', 'Jun', 'Jul', 'Aug',
        'Sep', 'Oct', 'Nov', 'Dec'
    ]
```

```
In [54]: plt.figure(figsize=(12,6))

sns.barplot(
    data=monthly_aqi,
    x='Month_Name',
    y='AQI'
)

plt.title('Average AQI by Month')
plt.xlabel('Month')
plt.ylabel('Average AQI')

plt.show()
```



```
In [55]: def get_season(month):

    if month in [12,1,2]:
        return 'Winter'

    elif month in [3,4,5]:
        return 'Summer'

    elif month in [6,7,8,9]:
        return 'Monsoon'

    else:
        return 'Post-Monsoon'
```

```
In [56]: aqi_clean['Season'] = aqi_clean['Month'].apply(get_season)
```

```
In [57]: seasonal_aqi = (
    aqi_clean
    .groupby('Season')['AQI']
    .mean()
    .reset_index()
)

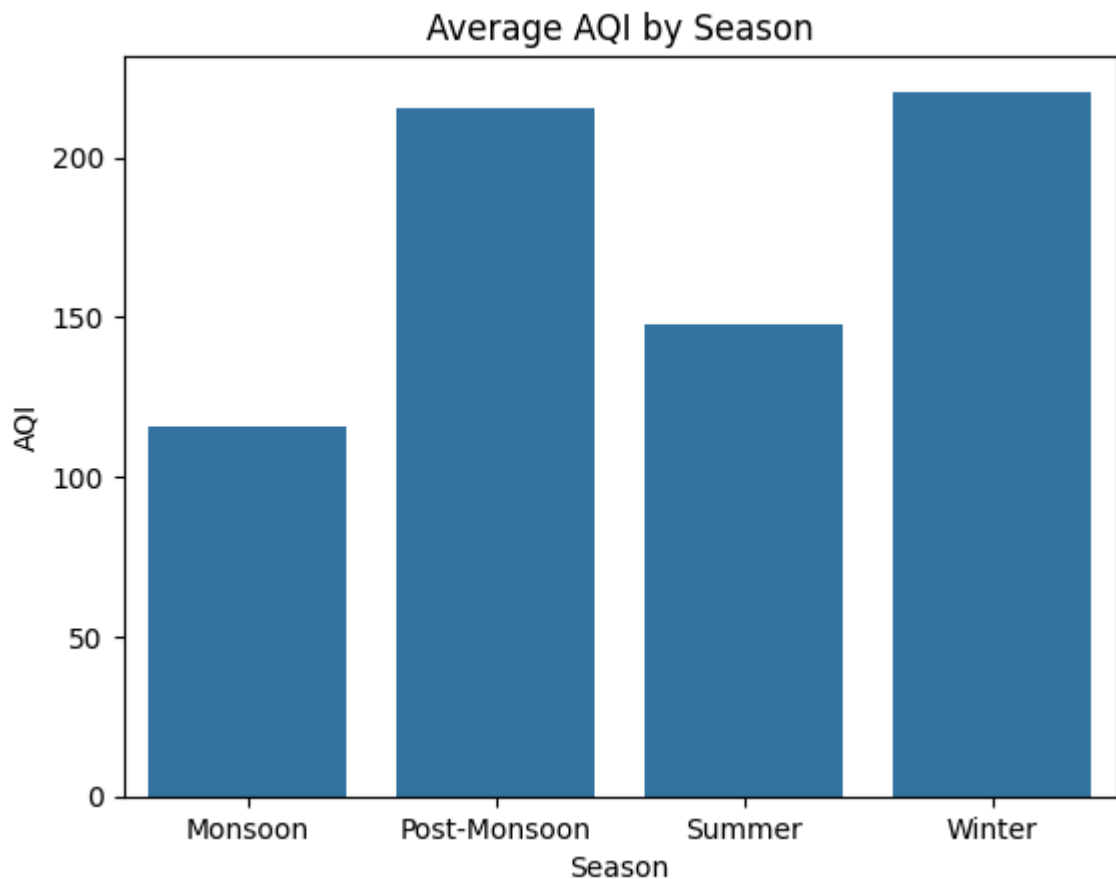
seasonal_aqi
```

```
Out[57]:
```

	Season	AQI
0	Monsoon	115.555426
1	Post-Monsoon	215.473464
2	Summer	147.670392
3	Winter	220.610764

```
In [58]: sns.barplot(
    data=seasonal_aqi,
    x='Season',
    y='AQI'
```

```
)
plt.title('Average AQI by Season')
plt.show()
```



```
In [59]: highest_month = monthly_aqi.loc[
          monthly_aqi['AQI'].idxmax()
        ]

print(highest_month)
```

```
Month          11
AQI          241.681302
Month_Name      Nov
Name: 10, dtype: object
```

```
In [60]: lowest_month = monthly_aqi.loc[
          monthly_aqi['AQI'].idxmin()
        ]

print(lowest_month)
```

```
Month          7
AQI          111.854575
Month_Name      Jul
Name: 6, dtype: object
```

To investigate whether air quality follows a seasonal pattern, the Date column was converted into a monthly time component and the average AQI was calculated for each month. A monthly comparison revealed clear seasonal variation in air quality throughout the year.

The analysis shows that November recorded the highest average AQI (241.68), while July recorded the lowest average AQI (111.85). Since November falls within the October–December harvest period identified by the NGO, the results provide evidence supporting the claim that air quality tends to be worse during this season.

Task 8

We must convert AQI from: City-Day to State-Year because crop data already exists at State-Year level.

```
In [61]: aqi_clean['Year'] = pd.to_datetime(
          aqi_clean['Date']
        ).dt.year
```

```
In [62]: aqi_state_year = (
          aqi_clean
          .groupby(['State', 'Year'])['AQI']
          .mean()
          .reset_index()
        )

aqi_state_year.head()
```

```
Out[62]:
```

	State	Year	AQI
0	Andhra Pradesh	2016	103.976048
1	Andhra Pradesh	2017	152.423469
2	Andhra Pradesh	2018	112.340125
3	Andhra Pradesh	2019	111.578462
4	Andhra Pradesh	2020	72.110803

```
In [63]: crop_state_year = (
          crop
          .groupby(['State_Name', 'Crop_Year'])
          .agg({
              'Area': 'sum',
              'Production': 'sum'
          })
          .reset_index()
        )

crop_state_year.head()
```

Out[63]:

	State_Name	Crop_Year	Area	Production
0	Andaman And Nicobar Islands	2000	44518.00	89060914.00
1	Andaman And Nicobar Islands	2001	41163.00	89718700.00
2	Andaman And Nicobar Islands	2002	45231.40	94387137.67
3	Andaman And Nicobar Islands	2003	44799.40	95296454.67
4	Andaman And Nicobar Islands	2004	45308.77	87186497.63

```
In [64]: merged = pd.merge(
    aqi_state_year,
    crop_state_year,
    left_on=['State', 'Year'],
    right_on=['State_Name', 'Crop_Year'],
    how='inner'
)
```

In [86]: merged.shape

Out[86]: (0, 7)

In [82]: crop['Crop_Year'].value_counts().sort_index()

```
Out[82]: Crop_Year
1997      8899
1998     11533
1999     12515
2000     13658
2001     13361
2002     16671
2003     17287
2004     14117
2005     13799
2006     14328
2007     14526
2008     14550
2009     14116
2010     14065
2011     14071
2012     13410
2013     13650
2014     10973
2015         562
Name: count, dtype: int64
```

In [87]: crop[crop['Crop_Year']==2015]['State_Name'].unique()

Out[87]: array(['Odisha', 'Sikkim'], dtype=object)

Your crop dataset has: 2015: Odisha Sikkim

While AQI dataset in 2015 has:

Bihar Delhi Gujarat Karnataka Tamil Nadu Telangana Uttar Pradesh

No overlap.

Therefore:

`pd.merge(...)` correctly returns: 0 rows because there are no matching State-Year combinations. The datasets cannot be merged reliably because the required overlap does not exist.

Task 9

Respected Minister,

Based on the analysis of air quality data across Indian cities, three important findings were found. First, most cities experience moderate to poor air quality, and pollution is not limited to a few isolated locations. Second, air pollution follows a clear seasonal pattern. Pollution levels were highest in November and lowest in July, indicating that air quality tends to worsen during the post-harvest and winter period. Third, several cities showed high pollution readings. This may affect public health and agricultural activities.

Some cities experience much more severe pollution episodes than others. So different measures for different cities are required.

A small number of extremely polluted days significantly increased the average AQI. So average AQI may not be reliable.

I recommend pollution-control measures during the post-harvest season, including monitoring pollution hotspots.

One limitation, the available air quality and crop production datasets could not be fully combined because they covered different states and years. Therefore, this analysis cannot prove whether poor air quality directly reduces crop production. Additional data is needed to confirm that relationship.